7.2.5 MLD ACS Extended Capability

CXL.io Requests and Completions are routed to the USP.

Table 7-9. MLD ACS Extended Capability

Register/Capability Structure	Capability Register Fields	FM-owned PPB	All vPPBs Bound to the MLD Port
ACS Capability Register	All fields	Supported	Supported because a vPPB can be bound to any port type
ACS Control Register	ACS Source Validation Enable	Hardwire to 0	Read/Write with no effect
	ACS Translation Blocking Enable	Hardwire to 0	Read/Write with no effect
	ACS P2P Request Redirect Enable	Hardwire to 1	Read/Write with no effect
	ACS P2P Completion Redirect Enable	Hardwire to 1	Read/Write with no effect
	ACS Upstream Forwarding Enable	Hardwire to 0	Read/Write with no effect
	ACS P2P Egress Control Enable	Hardwire to 0	Read/Write with no effect
	ACS Direct Translated P2P Enable	Hardwire to 0	Read/Write with no effect
	ACS I/O Request Blocking Enable	Hardwire to 0	Read/Write with no effect
	ACS DSP Memory Target Access Control	Hardwire to 0s	Read/Write with no effect
	ACS Unclaimed Request Redirect Control	Hardwire to 0	Read/Write with no effect

7.2.6 MLD PCIe Extended Capabilities

All fields in the PCIe Extended Capability structures for a vPPB shall behave identically to PCIe.

7.2.7 MLD Advanced Error Reporting Extended Capability

AER in an MLD port is separated into Triggering, Notifications, and Reporting. Triggering and AER Header Logging is handled at switch ingress and egress using switch-vendor-specific means. Notification is also switch-vendor specific, but it results in the vPPB logic for all vPPBs that are bound to the MLD port being informed of the AER errors that have been triggered. The vPPB logic is responsible for generating the AER status and error messages for each vPPB based on the AER Mask and Severity registers.

vPPBs that are bound to an MLD port support all the AER Mask and Severity configurability; however, some of the Notifications are suppressed to avoid confusion.

The PPB has its own AER Mask and Severity registers and the FM is notified of error conditions based on the Event Notification settings.

Errors that are not vPPB specific are provided to the host with a header log containing all 1s data. The hardware header log is provided only to the FM through the PPB.

Table 7-10 lists the AER Notifications and their routing indications for PPBs and vPPBs.

Table 7-10. MLD Advanced Error Reporting Extended Capability

Hardware Triggers	AER Error	FM-owned PPB	All vPPBs Bound to the MLD Port
AER Notifications	Data Link Protocol Error	Supported	Supported per vPPB
	Surprise Down Error	Supported	Supported per vPPB
	Poisoned TLP Received	Supported	Hardwire to 0
	Flow Control Protocol Error	Supported	Supported per vPPB
	Completer Abort	Supported	Supported to the vPPB that generated it
	Unexpected Completion	Supported	Supported to the vPPB that received it
	Receiver Overflow	Supported	Supported per vPPB
	Malformed TLP	Supported	Supported per vPPB
	ECRC Error	Supported	Hardwire to 0
	Unsupported Request	Supported	Supported per vPPB
	ACS Violation	Supported	Hardwire to 0
	Uncorrectable Internal Error	Supported	Supported per vPPB
	MC[1] Blocked	Supported	Hardwire to 0
	Atomic Op Egress Block	Supported	Hardwire to 0
	E2E TLP Prefix Block	Supported	Hardwire to 0
	Poisoned TLP Egress block	Supported	Hardwire to 0
	Bad TLP (correctable)	Supported	Supported per vPPB
	Bad DLLP (correctable)	Supported	Supported per vPPB
	Replay Timer Timeout (correctable)	Supported	Supported per vPPB
	Replay Number Rollover (correctable)	Supported	Supported per vPPB
	Other Advisory Non-Fatal (correctable)	Supported	Supported per vPPB
	Corrected Internal Error Status (correctable)	Supported	Supported per vPPB
	Header Log Overflow Status (correctable)	Supported	Supported per vPPB

1. Refers to Multicast.

7.2.8 MLD DPC Extended Capability

Downstream Port Containment has special behavior for an MLD Port. The FM configures the AER Mask and Severity registers in the PPB and also configures the AER Mask and Severity registers in the FMLD in the pooled device. As in an SLD port an unmasked uncorrectable error detected in the PPB and an ERR_NONFATAL and/or ERR_FATAL received from the FMLD can trigger DPC.

Continuing the model of the ultimate receiver being the entity that detects and reports errors, the ERR_FATAL and ERR_NONFATAL messages sent by a Logical Device can trigger a virtual DPC in the PPB. When a virtual DPC is triggered, the switch discards all traffic received from and transmitted to that specific LD. The LD remains bound to the vPPB and the FM is also notified. Software triggered DPC also triggers virtual DPC on a vPPB.

When the DPC trigger is cleared the switch autonomously allows passing of traffic to/ from the LD. Reporting of the DPC trigger to the host is identical to PCIe.

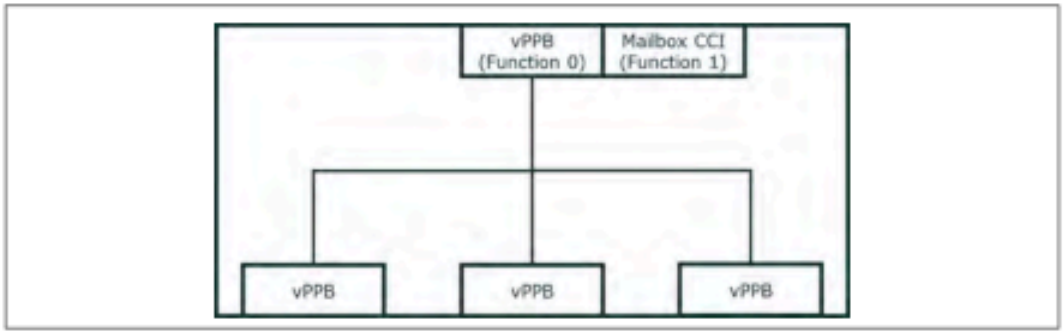
Table 7-11. MLD PPB DPC Extended Capability

Register/ Capability Structure	Capability Register Fields	FM-owned PPB	All vPPBs Bound to the MLD Port
DPC Control Register	DPC Trigger Enable	Supported	Switch internally detected unmasked uncorrectable errors do not trigger virtual DPC
	DPC Trigger Reason	Supported	Unmasked uncorrectable error is not a valid value

7.2.9 Switch Mailbox CCI

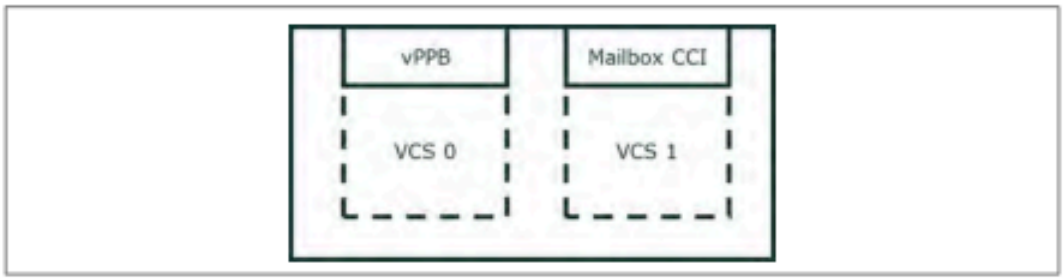
CXL Switch Mailbox CCIs optional. They are exposed as PCIe Endpoints with a Type 0 configuration space. In Single VCS and Multiple VCS, the Mailbox CCI is optional. If implemented, the Mailbox CCI shall be exposed to the Host in one of two possible configurations. In the first, it is exposed as an additional PCIe function in the Upstream Switch Port, as illustrated in Figure 7-15.

Figure 7-15. Multi-function Upstream vPPB



Switch Mailbox CCIs may also be exposed in a VCS with no vPPBs. In this configuration, the Mailbox CCI device is the only PCIe function that is present in the Upstream Port, as illustrated in Figure 7-16.

Figure 7-16. Single-function Mailbox CCI



7.3 CXL.io, CXL.cachemem Decode and Forwarding

7.3.1 CXL.io

Within a VCS, the CXL.io traffic must obey the same request, completion, address decode, and forwarding rules for a Switch as defined in PCIe Base Specification. There are additional decode rules that are defined to support an eRCD connected to a switch (see Section 9.12.4).

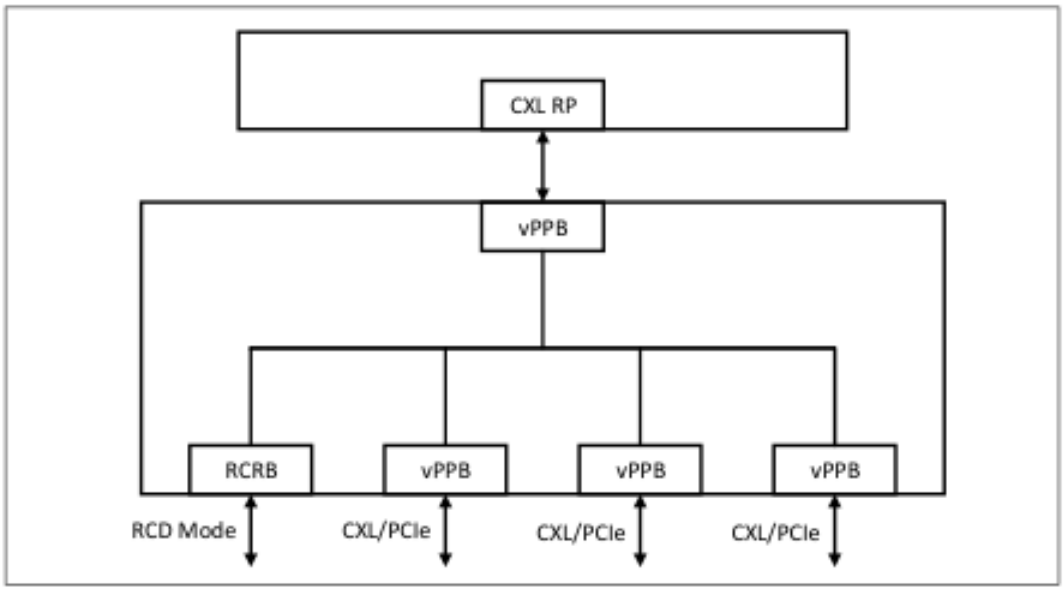
7.3.1.1 CXL.io Decode

When a TLP is decoded by a PPB, it determines the destination PPB to route the TLP based on the rules defined in PCIe Base Specification. Unless specified otherwise, all rules defined in PCIe Base Specification apply for routing of CXL.io TLPs. TLPs must be routed to PPBs within the same VCS. Routing of TLPs to and from an FM-owned PPB need to follow additional rules as defined in Section 7.2.3. P2P inside a Switch complex is limited to PPBs within a VCS.

7.3.1.2 RCD Support

RCDs are not supported behind ports that are configured to operate as FM-owned PPBs. When connected behind a switch, RCDs must appear to software as RCiEP devices. The mechanism defined in this section enables this functionality.

Figure 7-17. CXL Switch with a Downstream Link Auto-negotiated to Operate in RCD Mode



The CXL Extensions DVSEC for Ports (see Section 8.1.5) defines the alternate MMIO and Bus Range Decode windows for forwarding of requests to eRCDs connected behind a Downstream Port.

7.3.2 CXL.cache

If the switch does not support CXL.cache protocol enhancements that enable multi-device scaling (as described in Section 8.2.4.28), only one of the CXL SLD ports in the VCS is allowed to be enabled to support Type 1 devices or Type 2 devices. Requests and Responses received on the USP are routed to the associated DSP and vice-versa. Therefore, additional decode registers are not required for CXL.cache for such switches.

If the switch supports CXL.cache protocol enhancements that enable multi-device scaling, more than one of the CXL SLD ports in the VCS can be configured to support Type 1 devices or Type 2 devices. Section 9.15.2 and Section 9.15.3 describe how such a CXL switch routes CXL.cache traffic.

CXL.cache is not supported over FM-owned PPBs.

7.3.2.1 CXL.Cache Reserved bit forwarding

A switch shall forward 256B Flit messages reserved bits between the ingress port and the egress port. Both HBR and PBR formats are defined for 256B flit messages where a switch can translate between those formats. When performing the translation between HBR and PBR formats defined for 256B flits the Reserved bits shall be preserved. When a switch with 256B flit capability sends to a port with 68B flit format the Reserved bits shall be set to zero. Similarly, messages received as 68B flit formats shall never have reserved bits forwarded to a port with 256B flit messages.

Note: The reason for forwarding of reserved bits is to allow new features to be supported without requiring changes to existing switches. The reason for not forwarding in 68B flit format is that new features are expected to be added only to 256B flit formats so there is no need to support the complexity of translating reserved bits to/from 68B flits.

7.3.3 CXL.mem

The HDM Decode DVSEC capability contains registers that define the Memory Address Decode Ranges for Memory. CXL.mem requests originate from the Host/RP and flow downstream to the Devices through the switch. CXL.mem responses originate from the Device and flow upstream to the RP.

7.3.3.1 CXL.mem Request Decode

All CXL.mem Requests received by the USP target one of the Downstream PPBs within the VCS. The address decode registers in the VCS determine the downstream VCS PPB to route the request. The VCS PPB may be a VCS-owned PPB or an FM-owned PPB. See Section 7.3.4 for additional routing rules.

7.3.3.2 CXL.mem Response Decode

CXL.mem Responses received by the DSP target one and only one Upstream Port. For VCS-owned PPB the responses are routed to the Upstream Port of that VCS. Responses received on an FM-owned PPB go through additional decode rules to determine the VCS ID to route the requests to. See Section 7.3.4 for additional routing rules.

7.3.3.3 CXL.Mem Reserved bit forwarding

CXL.mem follows the same rules as CXL.cache as defined in Section 7.3.2.1.

7.3.4 FM-owned PPB CXL Handling

All PPBs are FM-owned. A PPB can be connected to a port that is disconnected or linked up. SLD components can be bound to a host or unbound. Unbound SLD components can be accessed by the FM using CXL.io transactions via the FM API. LDs within an MLD component can be bound to a host or unbound. Unbound LDs are FM-owned and can be accessed through the switch using CXL.io transactions via the FM API.

For all CXL.io transactions driven by the FM API, the switch acts as a virtual Root Complex for PPBs and Endpoints. The switch is responsible for enumerating the functions associated with that port and sending/receiving CXL.io traffic.

7.4 CXL Switch PM

7.4.1 CXL Switch ASPM L1

ASPM L1 for switch Ports is as defined in Chapter 10.0.

7.4.2 CXL Switch PCI-PM and L2

A vPPB in a VCS operates the same as a PCIe vPPB for handling of PME messages.

7.4.3 CXL Switch Message Management

CXL VDMs are of the "Local - Terminate at Receiver" type. When a switch is present in the hierarchy, the switch implements the message aggregation function and therefore all Host-generated messages terminate at the switch. The switch aggregation function is responsible for regenerating these messages on the Downstream Port. All messages and responses generated by the directly attached CXL components are aggregated and consolidated by the DSP and consolidated messages or responses are generated by the USP.

The PM message credit exchanges occur between the Host and Switch Aggregation port, and separately between the Switch Aggregation Port and device.

Table 7-12. CXL Switch Message Management

Message Type	Type	Switch Message Aggregation and Consolidation Responsibility
PM Reset Messages	Host Initiated	Host-generated requests terminate at Upstream Port, broadcast messages to all ports within VCS hierarchy
Sx Entry		
GPF Phase 1 Request		
GPF Phase 2 Request		
PM Reset Acknowledge	Device Responses	Device-generated responses terminate at Downstream Port within VCS hierarchy. Switch aggregates responses from all other connected ports within VCS hierarchy.
Sx Entry		
GPF Phase 1 Response		
GPF Phase 2 Response		

7.5 CXL Switch RAS

Table 7-13. CXL Switch RAS (Sheet 1 of 2)

Triggering Action	Description	Switch Action for Non-pooled Devices	Switch Action for Pooled Devices
Switch boot	Optional power-on reset pin	Assert PERST# Deassert PERST#	Assert PERST# Deassert PERST#
Upstream PERST# assert	VCS fundamental reset	Send Hot Reset	Write to MLD DVSEC to trigger LD Hot Reset of the associated LD **Note:** Only the FMLD provides the MLD DVSEC capability.
FM issues port reset command	Reset of an FM-owned DSP	Send Hot Reset	Send Hot Reset
PPB Secondary Bus Reset	Reset of an FM-owned DSP	Send Hot Reset	Write to MLD DVSEC to trigger LD Hot Reset of all LDs

Table 7-13. CXL Switch RAS (Sheet 2 of 2)

Triggering Action	Description	Switch Action for Non-pooled Devices	Switch Action for Pooled Devices
USP receives Hot Reset	VCS fundamental reset	Send Hot Reset	Write to MLD DVSEC to trigger LD Hot Reset of the associated LD
USP vPPB Secondary Bus Reset	VCS US SBR	Send Hot Reset	Write to MLD DVSEC to trigger LD Hot Reset of the associated LD
DSP vPPB Secondary Bus Reset	VCS DS SBR	Send Hot Reset	Write to MLD DVSEC to trigger LD Hot Reset of the associated LD
Host writes FLR	Device FLR	No switch involvement	No switch involvement
Switch watchdog timeout	Switch fatal error	Equivalent to power-on reset	Equivalent to power-on reset

Because the MLD DVSEC only exists in the FMLD, the switch must use the FM LD-ID in the CXL.io configuration write transaction when triggering LD reset.

7.6 Fabric Manager Application Programming Interface

This section describes the Fabric Manager Application Programming Interface.

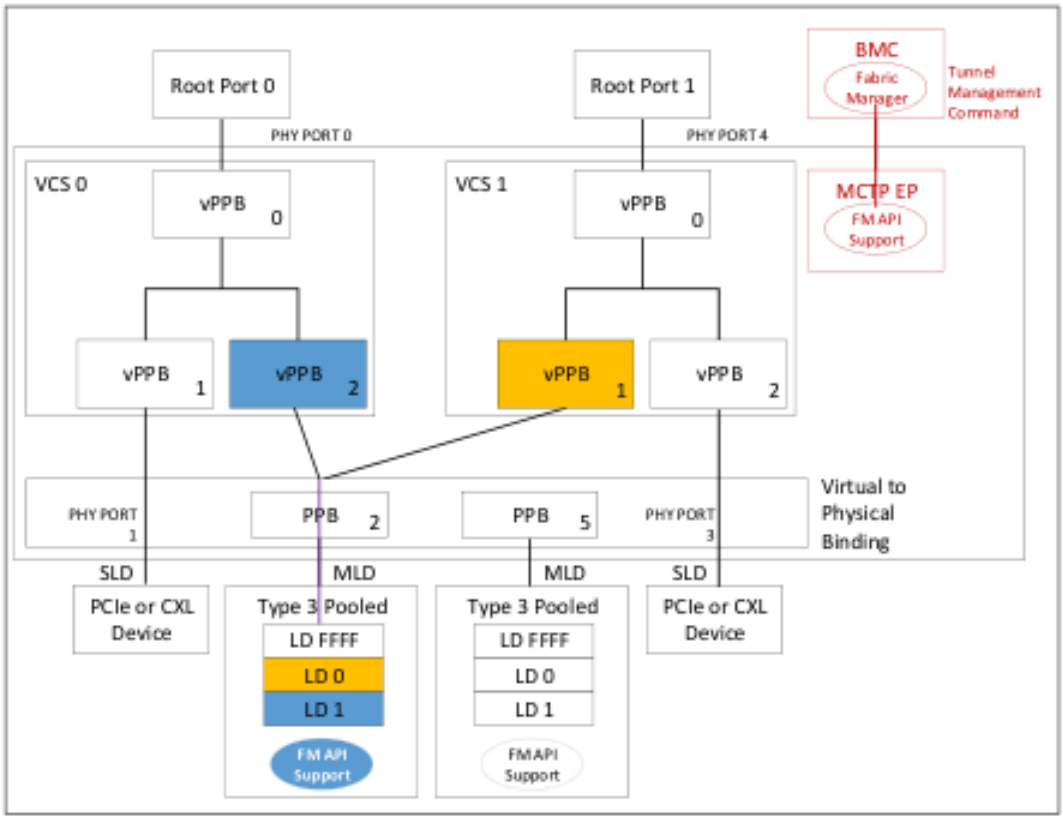
7.6.1 CXL Fabric Management

CXL devices can be configured statically or dynamically via a Fabric Manager (FM), an external logical process that queries and configures the system's operational state using the FM commands defined in this specification. The FM is defined as the logical process that decides when reconfiguration is necessary and initiates the commands to perform configurations. It can take any form, including, but not limited to, software running on a host machine, embedded software running on a BMC, embedded firmware running on another CXL device or CXL switch, or a state machine running within the CXL device itself.

7.6.2 Fabric Management Model

CXL devices are configured by FMs through the Fabric Manager Application Programming Interface (FM API) command sets, as defined in Section 8.2.10.10, through a CCI. A CCI is exposed through a device's Mailbox registers (see Section 8.2.9.4) or through an MCTP-capable interface. See Section 9.19 for details on the CCI processing of these commands.

Figure 7-18. Example of Fabric Management Model



FMs issue request messages and CXL devices issue response messages. CXL components may also issue the "Event Notification" request if notifications are supported by the component and the FM has requested notifications from the component using the Set MCTP Event Interrupt Policy command. See Section 7.6.3 for transport protocol details.

The following list provides a number of examples of connectivity between an FM and a component's CCI, but should not be considered a complete list:

- An FM directly connected to a CXL device through any MCTP-capable interconnect can issue FM commands directly to the device. This includes delivery over MCTP-capable interfaces such as SMBus as well as VDM delivery over a standard PCIe tree topology where the responder is mapped to a CXL attached device.
- An FM directly connected to a CXL switch may use the switch to tunnel FM commands to MLD components directly attached to the switch. In this case, the FM issues the "Tunnel Management Command" command to the switch specifying the switch port to which the device is connected. Responses are returned to the FM by the switch. In addition to MCTP message delivery, the FM command set provides the FM with the ability to have the switch proxy config cycles and memory accesses to a Downstream Port on the FM's behalf.
- An FM or part of the overall FM functionality may be embedded within a CXL component. The communication interface between such an embedded FM FW module and the component hardware is considered a vendor implementation detail and is not covered in this specification.

7.6.3 CCI Message Format and Transport Protocol

CCI commands are transmitted across MCTP-capable interconnects as MCTP messages using the format defined in Figure 7-19 and listed in Table 7-14.

Figure 7-19. CCI Message Format

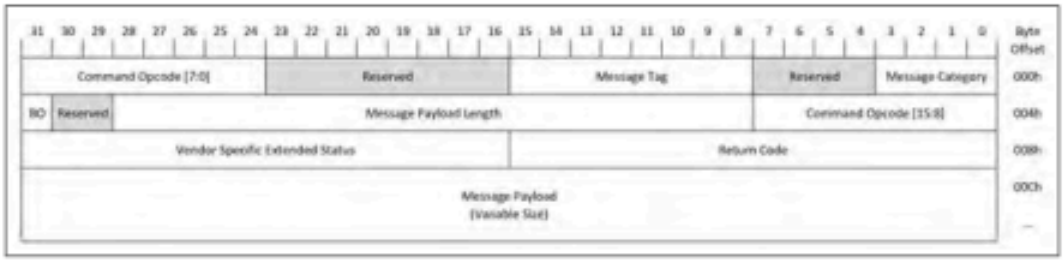


Table 7-14. CCI Message Format

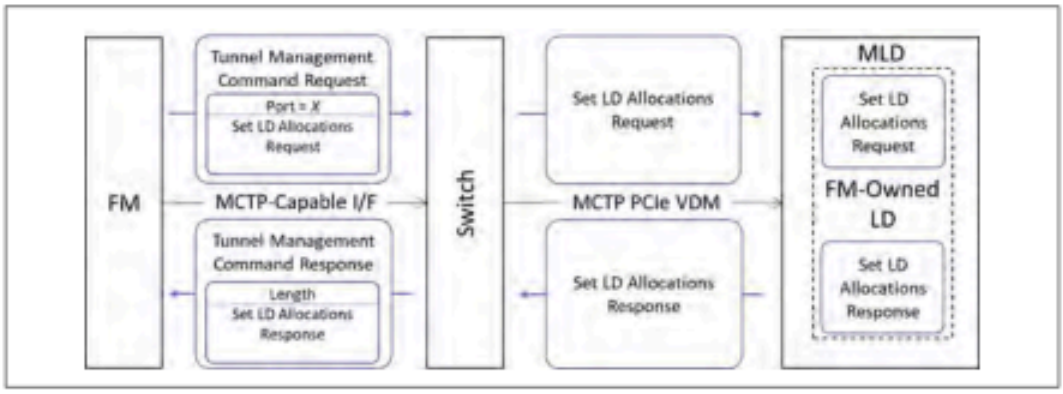
Byte Offset	Length in Bytes	Description
0h	1	• Bits[3:0]: **Message Category**: Type of CCI message: — 0h = Request — 1h = Response — All other encodings are reserved • Bits[7:4]: **Reserved**
1h	1	**Message Tag**: Tag number assigned to request messages by the Requester used to track response messages when multiple request messages are outstanding. Response messages shall use the tag number from the corresponding Request message.
2h	1	**Reserved**
3h	2	**Command Opcode[15:0]**: As defined in Table 8-49, Table 8-141, and Table 8-230.
5h	2	**Message Payload Length[15:0]**: Expressed in bytes. As defined in Table 8-49, Table 8-141, and Table 8-230.
7h	1	• Bits[4:0]: **Message Payload Length[20:16]**: Expressed in bytes. As defined in Table 8-49, Table 8-141, and Table 8-230. • Bits[6:5]: **Reserved**. • Bit[7]: **Background Operation (BO)**: As defined in Section 8.2.9.4.6.
8h	2	**Return Code[15:0]**: As defined in Table 8-46. Must be 0 for Request messages.
Ah	2	**Vendor Specific Extended Status[15:0]**: As defined in Section 8.2.9.4.6. Must be 0 for Request messages.
Ch	Varies	**Message Payload**: Variably sized payload for message in little-endian format. The length of this field is specified in the **Message Payload Length[20:0]** fields above. The format depends on **Opcode** and **Message Category**, as defined in Table 8-49, Table 8-141, and Table 8-230.

Commands from the FM API Command Sets may be transported as MCTP messages as defined in CXL Fabric Manager API over MCTP Binding Specification (DSP0234). All other CCI commands may be transported as MCTP messages as defined by the respective binding specification, such as CXL Type 3 Component Command Interface over MCTP Binding (DSP0281).

7.6.3.1 Transport Details for MLD Components

MLD components that do not implement an MCTP-capable interconnect other than their CXL interface shall expose a CCI through their CXL interface(s) using MCTP PCIe VDM Transport Binding Specification (DSP0238). FMs shall use the Tunnel Management Command to pass requests to the FM-owned LD, as illustrated in Figure 7-20.

Figure 7-20. Tunneling Commands to an MLD through a CXL Switch



7.6.4 CXL Switch Management

Dynamic configuration of a switch by an FM is not required for basic switch functionality, but is required to support MLDs or CXL fabric topologies.

7.6.4.1 Initial Configuration

The non-volatile memory of the switch stores, in a vendor-specific format, all necessary configuration settings that are required to prepare the switch for initial operation. This includes:

- Port configuration, including direction (upstream or downstream), width, supported rates, etc.
- Virtual CXL Switch configuration, including number of vPPBs for each VCS, initial port binding configuration, etc.
- CCI access settings, including any vendor-defined permission settings for management.

7.6.4.2 Dynamic Configuration

After initial configuration is complete and a CCI on the switch is operational, an FM can send Management Commands to the switch.

An FM may perform the following dynamic management actions on a CXL switch:

- Query switch information and configuration details
- Bind or Unbind ports
- Register to receive and handle event notifications from the switch (e.g., Hot-Plug, surprise removal, and failures)

When a switch port is connected to a downstream PCIe switch, and that port is bound to a vPPB, the management of that PCIe switch and its downstream device will be handled by the VCS's host, not the FM.

7.6.4.3 MLD Port Management

A switch with MLD Ports requires an FM to perform the following management activities:

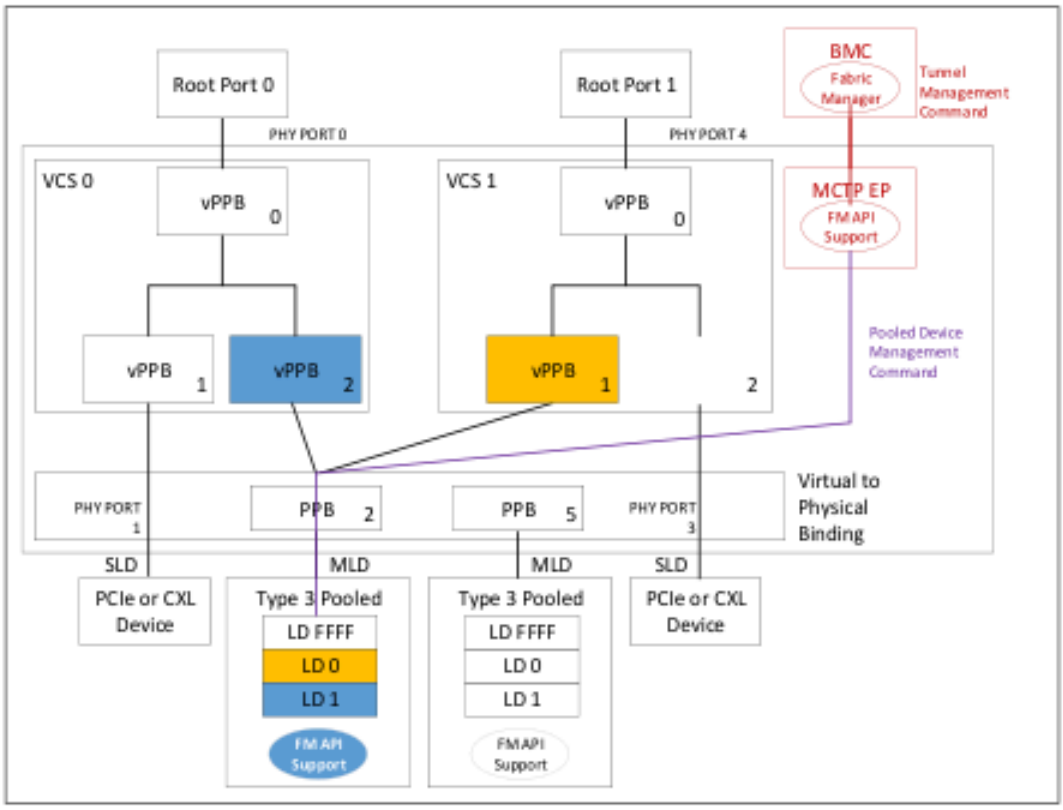
- MLD discovery
- LD binding/unbinding
- Management Command Tunneling

7.6.5 MLD Component Management

The FM can connect to an MLD over a direct connection or by tunneling its management commands through the CCI of the CXL switch to which the device is connected. The FM can perform the following operations:

- Memory allocation and QoS Telemetry management
- Security (e.g., LD erasure after unbinding)
- Error handling

Figure 7-21. Example of MLD Management Requiring Tunneling



7.6.6 Management Requirements for System Operations

This section presents examples of system use cases to highlight the role and responsibilities of the FM in system management. These use case discussions also serve to itemize the FM commands that CXL devices must support to facilitate each specific system behavior.

7.6.6.1 Initial System Discovery

As the CXL system initializes, the FM can begin discovering all direct attached CXL devices across all supported media interfaces. Devices supporting the FM API may be discovered using transport specific mechanisms such as the MCTP discovery process, as defined in MCTP Base Specification (DSP0236).

When a component is discovered, the FM shall issue the Identify command (see Section 8.2.10.1.1) prior to issuing any other commands to check the component's type and its maximum supported command message size. A return of "Retry Required" indicates that the component is not yet ready to accept commands. After receiving a successful response to the Identify request, the FM may issue the Set Response Message Limit command (see Section 8.2.10.1.4) to limit the size of response messages from the component based on the size of the FM's receive buffer. The FM shall not issue any commands with input arguments such that the command's response message exceeds the FM's maximum supported message size. Finally, the FM issues Get Log, as defined in Section 8.2.10.5.2.1, to read the Command Effects Log to determine which command opcodes are supported.

7.6.6.2 CXL Switch Discovery

After a CXL switch is released from reset (i.e., PERST# has been deasserted), it loads its initial configuration from non-volatile memory. Ports configured as DS PPBs will be released from reset to link up. Upon detection of a switch, the FM will query its configuration, capabilities, and connected devices. The **Physical Switch Command Set** is required for all switches implementing FM API support. The **Virtual Switch Command Set** is required for all switches that support multiple host ports.

An example of an FM Switch discovery process is as follows:

1. FM issues **Identify Switch Device** to check switch port count, enabled port IDs, number of supported LDs, and enabled VCS IDs.
2. FM issues **Get Physical Port State** for each enabled port to check port configuration (US or DS), link state, and attached device type. This allows the FM to check for any port link-up issues and create an inventory of devices for binding. If any MLD components are discovered, the FM can begin MLD Port management activities.
3. If the switch supports multiple host ports, FM issues **Get Virtual CXL Switch Info** for each enabled VCS to check for all bound vPPBs in the system and create a list of binding targets.

7.6.6.3 MLD and Switch MLD Port Management

MLDs must be connected to a CXL switch to share their LDs among VCSs. If an MLD is discovered in the system, the FM will need to prepare it for binding. A switch must support the **MLD Port Command Set** to support the use of MLDs. All MLD components shall support the MLD Component Command Set.

1. FM issues management commands to the device's LD FFFFh using **Tunnel Management Command**.
2. FM can execute advanced or vendor-specific management activities, such as encryption or authentication, using the **Send LD CXL.io Configuration Request** and **Send LD CXL.io Memory Request** commands.

7.6.6.4 Event Notifications

Events can occur on both devices and switches. The event types and records are listed in Section 7.6.8 for FM API events and in Section 8.2.10.2 for component events. The Event Records framework is defined in Section 8.2.10.2.1 to provide a standard event

record format that all CXL components shall use when reporting events to the managing entity. The managing entity specifies the notification method, such as MSI/MSI-X, EFN VDM, or MCTP Event Notification. The Event Notification message can be signaled by a device or by a switch; the notification always flows toward the managing entity. An Event Record is not sent with the Event Notification message. After the managing entity knows that an event has occurred, the entity can use component commands to read the Event Record.

1. To facilitate some system operations, the FM requires event notifications so it can execute its role in the process in a timely manner (e.g., notifying hosts of an asserted Attention Button on an MLD during a Managed Hot-Removal). If supported by the device, the FM can check and modify the current event notification settings with the Events command set.
2. If supported by the device, the event logs can be read with the **Get Event Records** command to check for any error events experienced by the device that might impact normal operation.

7.6.6.5 Binding Ports and LDs on a Switch

Once all devices, VCSs, and vPPBs have been discovered, the FM can begin binding ports and LDs to hosts as follows:

1. FM issues the **Bind vPPB** command specifying a physical port, VCS ID and vPPB index to bind the physical port to the vPPB. An LD-ID must also be specified if the physical port is connected to an MLD. The switch is permitted to initiate a Managed Hot-Add if the host has already booted, as defined in Section 9.9.
2. Upon completion of the binding process, the switch notifies the FM by generating a **Virtual CXL Switch Event Record**.

7.6.6.6 Unbinding Ports and LDs on a Switch

The FM can unbind devices or LDs from a VCS with the following steps:

1. FM issues the **Unbind vPPB** command specifying a VCS ID and vPPB index to unbind the physical port from the vPPB. The switch initiates a Managed Hot-Remove or Surprise Hot-Remove depending on the command options, as defined in PCIe Base Specification.
2. Upon completion of the unbinding process, the switch will generate a **Virtual CXL Switch Event Record**.

7.6.6.7 Hot-Add and Managed Hot-Removal of Devices

When a device is Hot-Added to an unbound port on a switch, the FM receives a notification and is responsible for binding as described in the steps below:

1. The switch notifies the FM by generating **Physical Switch Event Records** as the Presence Detect sideband signal is asserted or when a Link Up is detected if the PPB does not support Presence Detect.
2. FM issues the **Get Physical Port State** command for the physical port that has linked up to discover the connected device type. The FM can now bind the physical port to a vPPB. If it's an MLD, then the FM can proceed with MLD Port management activities; otherwise, the device is ready for binding.

When a device is Hot-Removed from an unbound port on a switch, the FM receives a notification. The switch notifies the FM by generating **Physical Switch Event Records** as the Presence Detect sideband is deasserted and the associated port links down.

1. The switch notifies the FM by generating **Physical Switch Event Records** as the Presence Detect sideband is deasserted and the associated port links down.

When an SLD or PCIe device is Hot-Added to a bound port, the FM can be notified but is not involved. When a Surprise or Managed Hot-Removal of an SLD or PCIe device occurs on a bound port, the FM can be notified but is not involved.

A bound port will not advertise support for MLDs during negotiation, so MLD components will link up as an SLD.

The FM manages managed hot-removal of MLDs as follows:

1. When the Attention Button sideband is asserted on an MLD port, the Attention state bit is updated in the corresponding PPB and vPPB CSRs and the switch notifies the FM and hosts with LDs that are bound and below that MLD port. The hosts are notified with the MSI/MSI-X interrupts assigned to the affected vPPB and a **Virtual CXL Switch Event Record** is generated.
2. As defined in PCIe Base Specification, hosts will read the Attention State bit in their vPPB's CSR and prepare for removal of the LD. When a host is ready for the LD to be removed, it will set the Attention LED bit in the associated vPPB's CSR. The switch records these CSR updates by generating **Virtual CXL Switch Event Records**. The FM unbinds each assigned LD with the **Unbind vPPB** command as it receives notifications from each host.
3. When all host handshakes are complete, the MLD is ready for removal. The FM uses the **Send PPB CXL.io Configuration Request** command to set the Attention LED bit in the MLD port PPB to indicate that the MLD can be physically removed. The timeout value for the host handshakes to complete is implementation specific. There is no requirement for the FM to force the unbind operation, but it can do so using the "Simulate Surprise Hot-Remove" unbinding option in the **Unbind vPPB** command.

7.6.6.8 Surprise Removal of Devices

There are two kinds of surprise removals: physical removal of a device, and surprise Link Down. The main difference between the two is the state of the presence pin, which will be deasserted after a physical removal but will remain asserted after a surprise Link Down. The switch notifies the FM of a surprise removal by generating **Virtual CXL Switch Event Records** for the change in link status and Presence Detect, as applicable.

Three cases of Surprise Removal are described below:

- When a Surprise Removal of a device occurs on an unbound port, the FM must be notified.
- When a Surprise Removal of an SLD or PCIe device occurs on a bound port, the FM is permitted to be notified but must not be involved in any error handling operations.
- When a Surprise Removal of an MLD component occurs, the FM must be notified. The switch will automatically unbind any existing LD bindings. The FM must perform error handling and port management activities, the details of which are considered implementation specific.

7.6.7 Fabric Management Application Programming Interface

The FM manages all devices in a CXL system via the sets of commands defined in the FM API. This specification defines the minimum command set requirements for each device type.

Note: CXL switches and MLDs require FM API support to facilitate the advanced system capabilities outlined in Section 7.6.6. FM API is optional for all other CXL device types.

Table 7-15. FM API Command Sets

Command Set Name	HBR Switch FM API Requirement[1]	MLD FM API Requirement[1]
Physical Switch (Section 7.6.7.1)	M	P
Virtual Switch (Section 7.6.7.2)	O	P
MLD Port (Section 7.6.7.3)	O	P
MLD Component (Section 7.6.7.4)	P	M
Multi-Headed Device (Section 7.6.7.5)	P	P
DCD Management (Section 7.6.7.6)	P	O
PBR Switch (Section 7.7.13)	P	P
Global Memory Access Endpoint (Section 7.7.14)	P	P

1. M = Mandatory, O = Optional, P = Prohibited.

Command opcodes are listed in Table 8-230. Table 8-230 also identifies the minimum command sets and commands that are required to implement defined system capabilities. The following subsections define the commands grouped in each command set. Within each command set, commands are marked as mandatory (M) or optional (O). If a command set is supported, the required commands within that set must be implemented, but only if the Device supports that command set. For example, the Get Virtual CXL Switch Information command is required in the Virtual Switch Command Set, but that set is optional for switches. This means that a switch does not need to support the Get Virtual CXL Switch Information command if it does not support the Virtual Switch Command Set.

All commands have been defined as stand-alone operations; there are no explicit dependencies between commands, so optional commands can be implemented or not implemented on a per-command basis. Requirements for the implementation of commands are driven instead by the desired system functionality.

7.6.7.1 Physical Switch Command Set

This command set is only supported by and must be supported by CXL switches that have FM API support.

7.6.7.1.1 Identify Switch Device (Opcode 5100h)

This command retrieves information about the capabilities and configuration of a CXL switch.

Possible Command Return Codes:

- Success
- Internal Error
- Retry Required

Command Effects:

- None

Table 7-16. Identify Switch Device Response Payload

Byte Offset	Length in Bytes	Description
00h	1	**Ingress Port ID**: Ingress CCI port index of the received request message. For CXL/PCIe ports, this corresponds to the physical port number. For non-CXL/PCIe, this corresponds to a vendor-specific index of the buses that the device supports, starting at 0. For example, a request received on the second of 2 SMBuses supported by a device would return a 1.
01h	1	**Reserved**
02h	1	**Number of Physical Ports**: Total number of physical ports in the CXL switch, including inactive/disabled ports.
03h	1	**Number of VCSs**: Maximum number of virtual CXL switches that are supported by the CXL switch.
04h	20h	**Active Port Bitmask**: Bitmask that defines whether a physical port is enabled (1) or disabled (0). Each bit corresponds 1:1 with a port, with the least significant bit corresponding to Port 0.
24h	20h	**Active VCS Bitmask**: Bitmask that defines whether a VCS is enabled (1) or disabled (0). Each bit corresponds 1:1 with a VCS ID, with the least significant bit corresponding to VCS 0.
44h	2	**Total Number of vPPBs**: The number of virtual PPBs that are supported by the CXL switch across all VCSs.
46h	2	**Number of Bound vPPBs**: Total number of vPPBs, across all VCSs, that are bound.
48h	1	**Number of HDM Decoders**: Number of HDM decoders available per USP.

7.6.7.1.2 Get Physical Port State (Opcode 5101h)

This command retrieves the physical port information.

Possible Command Return Codes:

- Success
- Invalid Input
- Internal Error
- Retry Required

Command Effects:

- None

Table 7-17. Get Physical Port State Request Payload

Byte Offset	Length in Bytes	Description
0h	1	**Number of Ports**: Number of ports requested.
1h	Varies	**Port ID List**: 1-byte ID of requested port, repeated **Number of Ports** times.

Table 7-18. Get Physical Port State Response Payload

Byte Offset	Length in Bytes	Description
0h	1	**Number of Ports**: Number of port information blocks returned.
1h	3	**Reserved**
4h	Varies	**Port Information List**: Port information block as defined in Table 7-19, repeated **Number of Ports** times.

Table 7-19. Get Physical Port State Port Information Block Format (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
0h	1	**Port ID**
1h	1	• Bits[3:0]: **Current Port Configuration State**: — 0h = Disabled — 1h = Bind in progress — 2h = Unbind in progress — 3h = DSP — 4h = USP — 5h = Fabric Port — Fh = Invalid Port_ID; all subsequent field values are undefined — All other encodings are reserved • Bit[4]: **GAE Support**: Indicates whether GAE support is present (1) or not present (0) on a port. Valid only for PBR switches if Current Port Configuration State is 4h (USP). • Bits[7:5]: **Reserved**.
2h	1	• Bits[3:0]: **Connected Device Mode**: Formerly known as Connected Device CXL Version. This field is reserved for all values of Current Port Configuration State except DSP. — 0h = Connection is not CXL or is disconnected — 1h = RCD mode — 2h = CXL 68B Flit and VH mode — 3h = Standard 256B Flit mode — 4h = CXL Latency-Optimized 256B Flit mode — 5h = PBR mode — All other encodings are reserved • Bits[7:4]: **Reserved**.
3h	1	**Reserved**
4h	1	**Connected Device Type** • 00h = No device detected • 01h = PCIe Device • 02h = CXL Type 1 device • 03h = CXL Type 2 device or HBR switch • 04h = CXL Type 3 SLD • 05h = CXL Type 3 MLD • 06h = PBR component • All other encodings are reserved This field is reserved if **Supported CXL Modes** is 00h. This field is reserved for all values of **Current Port Configuration State** except 3h (DSP) or 5h (Fabric Port).
5h	1	**Supported CXL Modes**: Formerly known as Connected CXL Version. Bitmask that defines which CXL modes are supported (1) or not supported (0) by this port: • Bit[0]: **RCD Mode** • Bit[1]: **CXL 68B Flit and VH Capable** • Bit[2]: **256B Flit Capable** • Bit[3]: **CXL Latency-Optimized 256B Flit Capable** • Bit[4]: **PBR Capable** • Bits[7:5]: Reserved for future CXL use Undefined when the value is 00h.

Table 7-19. Get Physical Port State Port Information Block Format (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
6h	1	• Bits[5:0]: **Maximum Link Width**: Value encoding matches the Maximum Link Width field in the PCIe Link Capabilities register in the PCIe Capability structure. • Bits[7:6]: **Reserved**.
7h	1	• Bits[5:0]: **Negotiated Link Width**: Value encoding matches the Negotiated Link Width field in PCIe Link Capabilities register in the PCIe Capability structure. • Bits[7:6]: **Reserved**.
8h	1	• Bits[5:0]: **Supported Link Speeds Vector**: Value encoding matches the Supported Link Speeds Vector field in the PCIe Link Capabilities 2 register in the PCIe Capability structure. • Bits[7:6]: **Reserved**.
9h	1	• Bits[5:0]: **Max Link Speed**: Value encoding matches the Max Link Speed field in the PCIe Link Capabilities register in the PCIe Capability structure. • Bits[7:6]: **Reserved**.
Ah	1	• Bits[5:0]: **Current Link Speed**: Value encoding matches the Current Link Speed field in the PCIe Link Status register in the PCIe Capability structure. • Bits[7:6]: **Reserved**.
Bh	1	**LTSSM State**: Current link LTSSM Major state: • 00h = Detect • 01h = Polling • 02h = Configuration • 03h = Recovery • 04h = L0 • 05h = L0s • 06h = L1 • 07h = L2 • 08h = Disabled • 09h = Loopback • 0Ah = Hot Reset • All other encodings are reserved Link substates should be reported through vendor-defined diagnostics commands.
Ch	1	**First Negotiated Lane Number**
Dh	2	**Link State Flags** • Bit[0]: **Lane Reversal State**: — 0 = Standard lane ordering — 1 = Reversed lane ordering • Bit[1]: **Port PCIe Reset State (PERST#)**: — 0 = Not in reset — 1 = In reset • Bit[2]: **Port Presence Pin State (PRSNT#)**: — 0 = Not present — 1 = Present • Bit[3]: **Power Control State**: — 0 = Power on — 1 = Power off • Bits[15:4]: **Reserved**
Fh	1	**Supported LD Count**: Number of additional LDs supported by this port. All ports must support at least one LD.

7.6.7.1.3 Physical Port Control (Opcode 5102h)

This command is used by the FM to control unbound ports and MLD ports, including issuing resets and controlling sidebands.

Possible Command Return Codes:

- Success

- Unsupported
- Invalid Input
- Internal Error
- Retry Required

Command Effects:

- None

Table 7-20. Physical Port Control Request Payload

Byte Offset	Length in Bytes	Description
0h	1	**PPB ID**: Physical PPB ID, which corresponds 1:1 to associated physical port number.
1h	1	**Port Opcode**: Code that defines which operation to perform: • 00h = Assert PERST# • 01h = Deassert PERST# • 02h = Reset PPB • All other encodings are reserved

7.6.7.1.4 Send PPB CXL.io Configuration Request (Opcode 5103h)

This command sends CXL.io Config requests to the specified physical port's PPB. This command is only processed for unbound ports and MLD ports.

Possible Command Return Codes:

- Success
- Invalid Input
- Internal Error
- Retry Required

Command Effects:

- None

Table 7-21. Send PPB CXL.io Configuration Request Input Payload

Byte Offset	Length in Bytes	Description
0h	1	**PPB ID**: Target PPB's physical port.
1h	3	• Bits[7:0]: **Register Number**: As defined in PCIe Base Specification • Bits[11:8]: **Extended Register Number**: As defined in PCIe Base Specification • Bits[15:12]: **First Dword Byte Enable**: As defined in PCIe Base Specification • Bits[22:16]: **Reserved** • Bit[23]: **Transaction Type**: — 0 = Read — 1 = Write
4h	4	**Transaction Data**: Write data. Valid only for write transactions.

Table 7-22. Send PPB CXL.io Configuration Request Output Payload

Byte Offset	Length in Bytes	Description
0h	4	**Return Data**: Read data. Valid only for read transactions.

7.6.7.1.5 Get Domain Validation SV State (Opcode 5104h)

This command is used by the Host to check the state of the secret value.

Possible Command Return Codes:

- Success
- Internal Error
- Retry Required

Command Effects:

- None

Table 7-23. Get Domain Validation SV State Response Payload

Byte Offset	Length in Bytes	Description
0h	1	**Secret Value State**: State of the secret value: • 00h = Not set • 01h = Set • All other encodings are reserved

7.6.7.1.6 Set Domain Validation SV (Opcode 5105h)

This command is used by the Host to set the secret value of its VCS. The secret value can be set only once. This command will fail with Invalid Input if it is called more than once.

Possible Command Return Codes:

- Success
- Invalid Input
- Internal Error
- Retry Required

Command Effects:

- None

Table 7-24. Set Domain Validation SV Request Payload

Byte Offset	Length in Bytes	Description
0h	10h	**Secret Value**: UUID used to uniquely identify a host hierarchy.

7.6.7.1.7 Get VCS Domain Validation SV State (Opcode 5106h)

This command is used by the FM to check the state of the secret value in a VCS.

Possible Command Return Codes:

- Success
- Internal Error
- Retry Required

Command Effects:

- None